

Tema 2: Pilas y colas

2.1 Convenciones de C++

Durante el curso utilizaremos algunas buenas prácticas para escribir código seguro y legible. Estas convenciones se suman a los conceptos ya vistos anteriormente (`inline`, `static` o `template`).

- **Nombre de atributos privados (`_`):** Los atributos privados terminan con un guion bajo para diferenciarlos intuitivamente de los parámetros de fuera de la clase. Ej: `valor_`.
- **Lista de inicializadores (`:`):** Se utiliza en el `.cpp` para construir directamente los atributos de la clase antes de entrar a las llaves del constructor, mejorando el rendimiento respecto a asignarlo a posteriori.
- **Precondiciones (`assert`):** Detiene la ejecución del programa inmediatamente si una condición es falsa, evitando propagar errores silenciosos y alertando al programador de irregularidades de uso. Requiere `#include <cassert>`.

```
“cpp [Caja.cpp] #include “Caja.hh” #include
```

```
// Uso de la lista de inicializadores (:) y assert Caja::Caja(int valor_inicial) : valor_(valor_inicial) { as-  
sert(valor_inicial >= 0); // Detiene el programa si el valor es negativo }
```

```
*[Simulació interactiva disponible a la web]*
```

```
---
```

```
## 2.2 La pila (`stack`)
```

Estructura de datos lineal con política **LIFO** (Last In, First Out): el último elemento en entrar es el primero en salir.

```
### Implementación interna
```

A nivel interno encapsula un `std::vector` y restringe las operaciones exclusivamente a su extremo final.

```
**Ejemplo simplificado de interfaz <code>Stack.hh</code> propia**
```

```
```cpp
```

```
#ifndef STACK_HH
```

```
#define STACK_HH
```

```
#include <vector>
```

```
#include <cassert>
```

```
template <typename T>
```

```
class Stack {
```

```
 std::vector<T> elems_;
```

```

public:
 void push(const T& x) { elems_.push_back(x); }
 void pop() {
 assert(elems_.size() > 0);
 elems_.pop_back();
 }
 const T& top() const {
 assert(elems_.size() > 0);
 return elems_.back();
 }
 bool empty() const { return elems_.empty(); }
 int size() const { return elems_.size(); }
};
#endif

```

### Uso de <stack> de C++

La librería de C++ proporciona la clase ya fabricada `std::stack`. Todas estas operaciones tienen coste constante  $\mathcal{O}(1)$ :

- **push(x)**: Añade el valor `x` por encima (en la cima).
- **top()**: Consulta (devuelve) el elemento de la cima sin borrarlo.
- **pop()**: Elimina el elemento superior. **Solo lo elimina, no lo devuelve.**

```

#include <stack>
using namespace std;

stack<int> S;
S.push(10); // S: [10]
S.push(20); // S: [10, 20] <- Cima
int x = S.top(); // Nos guardamos x=20
S.pop(); // Extrae el 20. S: [10] <- Cima

```

*[Simulación interactiva disponible a la web]*

---

## 2.3 La cola (queue)

Estructura de datos lineal con política **FIFO (First In, First Out)**: el primero en entrar es el primero en salir. Se trata de una cola convencional (se llega por detrás, se extrae por delante).

### Implementación interna: La eficiencia

A diferencia de la pila, borrar metódicamente al principio de un `std::vector` tiene un elevado coste  $\mathcal{O}(n)$ , porque es obligatorio desplazar el resto de elementos para llenar el hueco de memoria. Por ello, la cola no utiliza un vector de base sino generalmente un `std::deque` (Double Ended Queue), un soporte optimizado que permite tanto inserciones por la derecha como extracciones por la izquierda en tiempo constante  $\mathcal{O}(1)$ .

### Ejemplo simplificado de interfaz Queue.hh propia usando deque

```

#ifndef QUEUE_HH
#define QUEUE_HH
#include <deque>
#include <cassert>

template <typename T>

```

```

class Queue {
 std::deque<T> elems_;

public:
 void push(const T& x) { elems_.push_back(x); }
 void pop() {
 assert(elems_.size() > 0);
 elems_.pop_front();
 }
 const T& front() const {
 assert(elems_.size() > 0);
 return elems_.front();
 }
 bool empty() const { return elems_.empty(); }
 int size() const { return elems_.size(); }
};
#endif

```

### Uso de <queue> de C++

Proporcionado mediante `std::queue`. Métodos  $\mathcal{O}(1)$ :

- **push(x)**: Añade `x` al final (ingresando fila).
- **front()**: Consulta el elemento de delante (el primero dispuesto para salir).
- **pop()**: Elimina y libera el elemento de delante (sin devolverlo).

```

#include <queue>
using namespace std;

queue<int> Q;
Q.push(10); // Q: delante[10]detrás
Q.push(20); // Q: delante[10, 20]detrás.
int x = Q.front(); // x=10, el que era el primero en la fila.
Q.pop(); // Extrae el elemento delantero. Q: delante[20]detrás.

```